CS240: Database Systems

Lecture 02:
ER Diagram

Outline

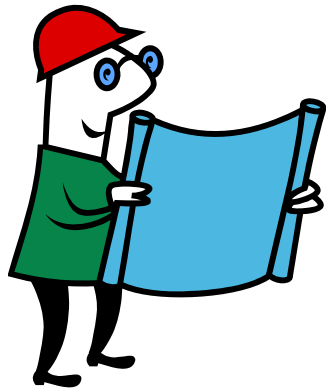
- **Database Design**
- Entity
- Relationship
  - Binary relationship
  - Non-binary relationship
- Weak Entity/Strong Entity
- Class Hierarchy
- Aggregation
- Conceptual Design with the E-R Model

Database Design

- The database design process can be divided into six steps.

1. Requirement Analysis
   - What data is to be stored?
   - What applications must be built.
   - What operations are the most frequent and subject to performance requirements.
   - The outcome of this phase is a specification of user requirements.

Database Design



2. Conceptual database design
   - High-level description of
     - Data to be stored
     - Constraints
   - Often carried out using the ER model
3. Logical database design
   - Choose a DBMS to implement our database design.
   - Convert the conceptual database design into a database schema in the data model of the chosen DBMS.
     - Convert ER schema into a relational database schema.

Database Design

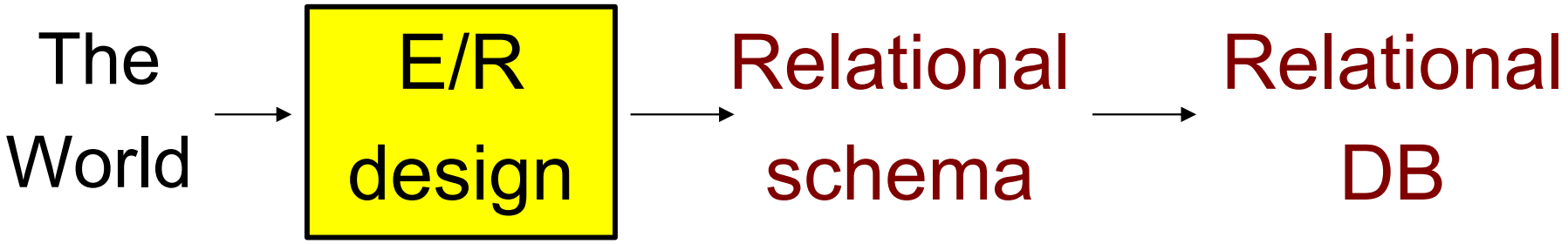
4. Schema Refinement
   - Analyze the collection of relations in our relational database.
   - Identify the potential problems.
   - Refine the schema

5. Physical database design
   - Ensure that the design meets the performance requirement.
   - Build index, clustering some tables etc.
   - Redesign parts of the database schema.

Database Design

6. Application and security design
   - Write application programs.
   - Identify data that can be accessible by certain types of users.
   - Take steps to ensure that access rules are enforced.

DB Development Path



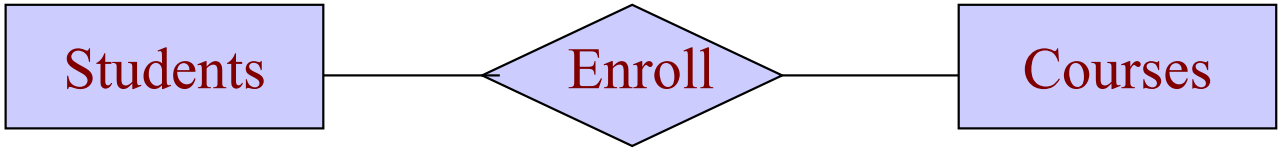
1. Requirement analysis
2. Conceptual database design – ER
3. Logical database design – relational schema
4. Schema refinement
5. Physical database design
6. Application and security design

Entity-Relationship Model

- **Entity-Relationship (ER) model** is a popular conceptual data model.
- This model is used in the design of database applications.
- E-R model views the real world as a collection of **entities** and **relationships** among entities.

Entity-Relationship Model

- **E/R design** translated to a relational design then implemented in an RDBMS
- Elements of model
  - Entities, Entity Sets, Attributes
  - Relationships **(!= relations!)**
- Graphical representation of miniworld
  - Entity: like an object, represented by a rectangle
  - Relationship: connect two or more entity sets, represented by diamonds



Outline

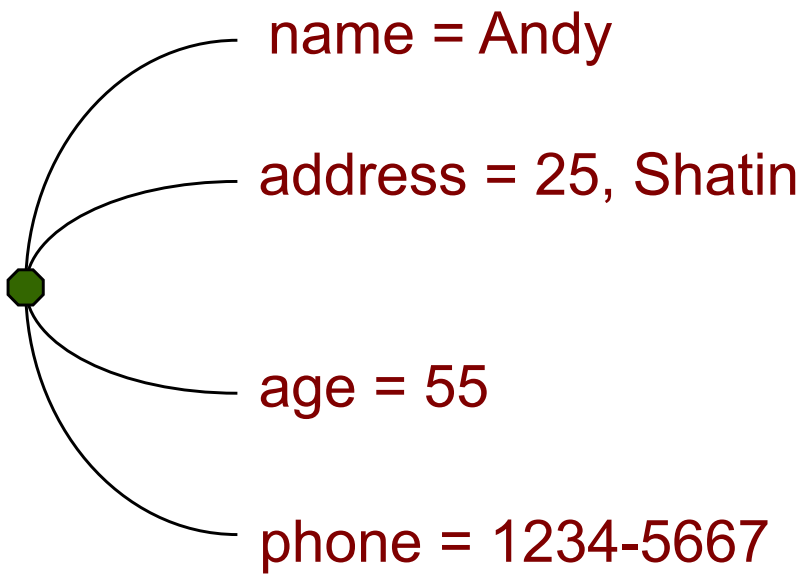
- Database Design
- **Entity**
- Relationship
  - Binary relationship
  - Non-binary relationship
- Weak Entity/Strong Entity
- Class Hierarchy
- Aggregation
- Conceptual Design with the E-R Model

Entities and Attributes

- E-R model views the real world as a collection of entities and relationships among entities.
- An entity is an object in the real world that is distinguishable from other objects.
  - Examples
    - A classroom
    - A teacher
    - The address of the teacher

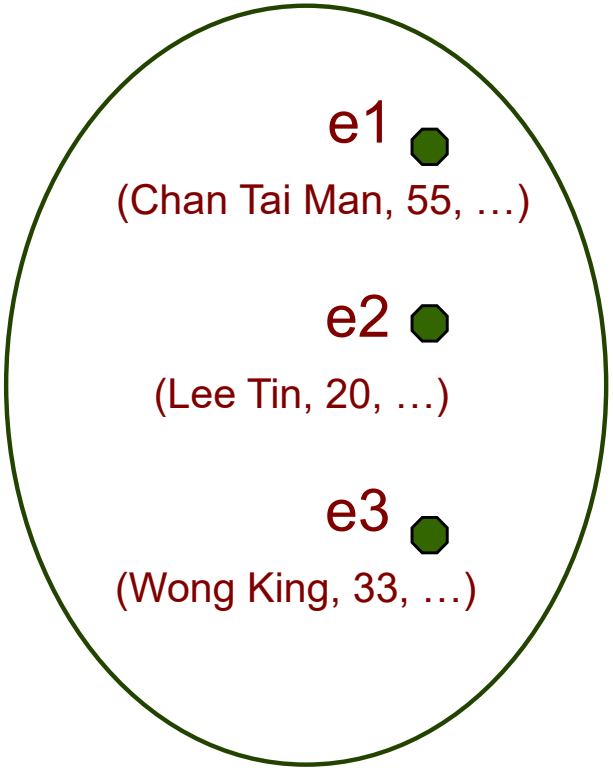
Entities and Attributes

- An entity is described using a set of attributes whose values are used to distinguish one entity from another of the same type
- Entity - Employee



Entities and Attributes

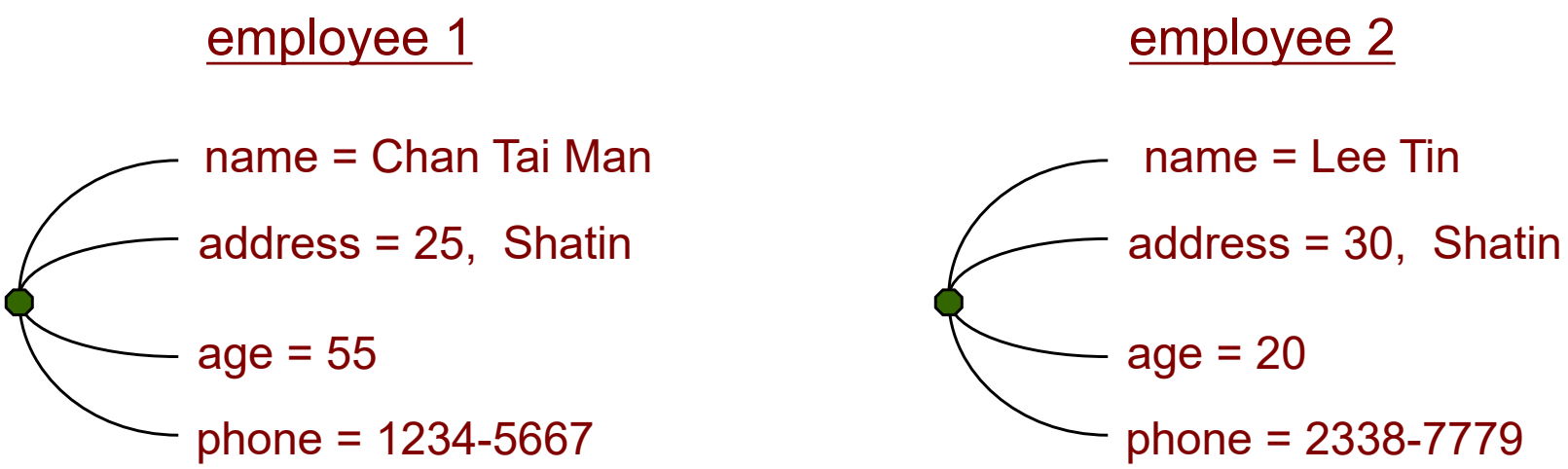
- An entity set is a collection of entities of the same type.



Entities and Attributes

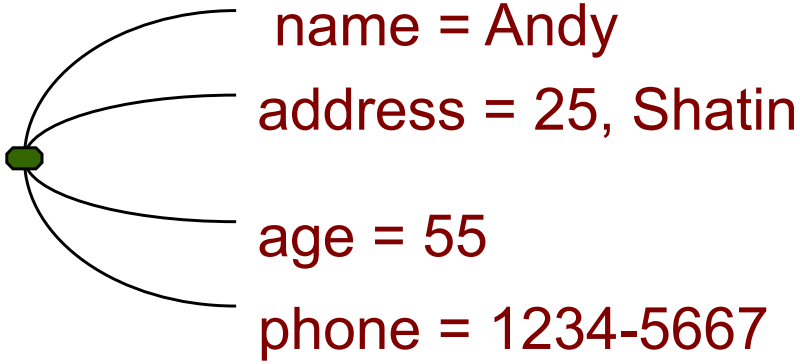
- All entities in a given entity set have the same attributes (the values may be different).

employee = (name, address, age, phone)



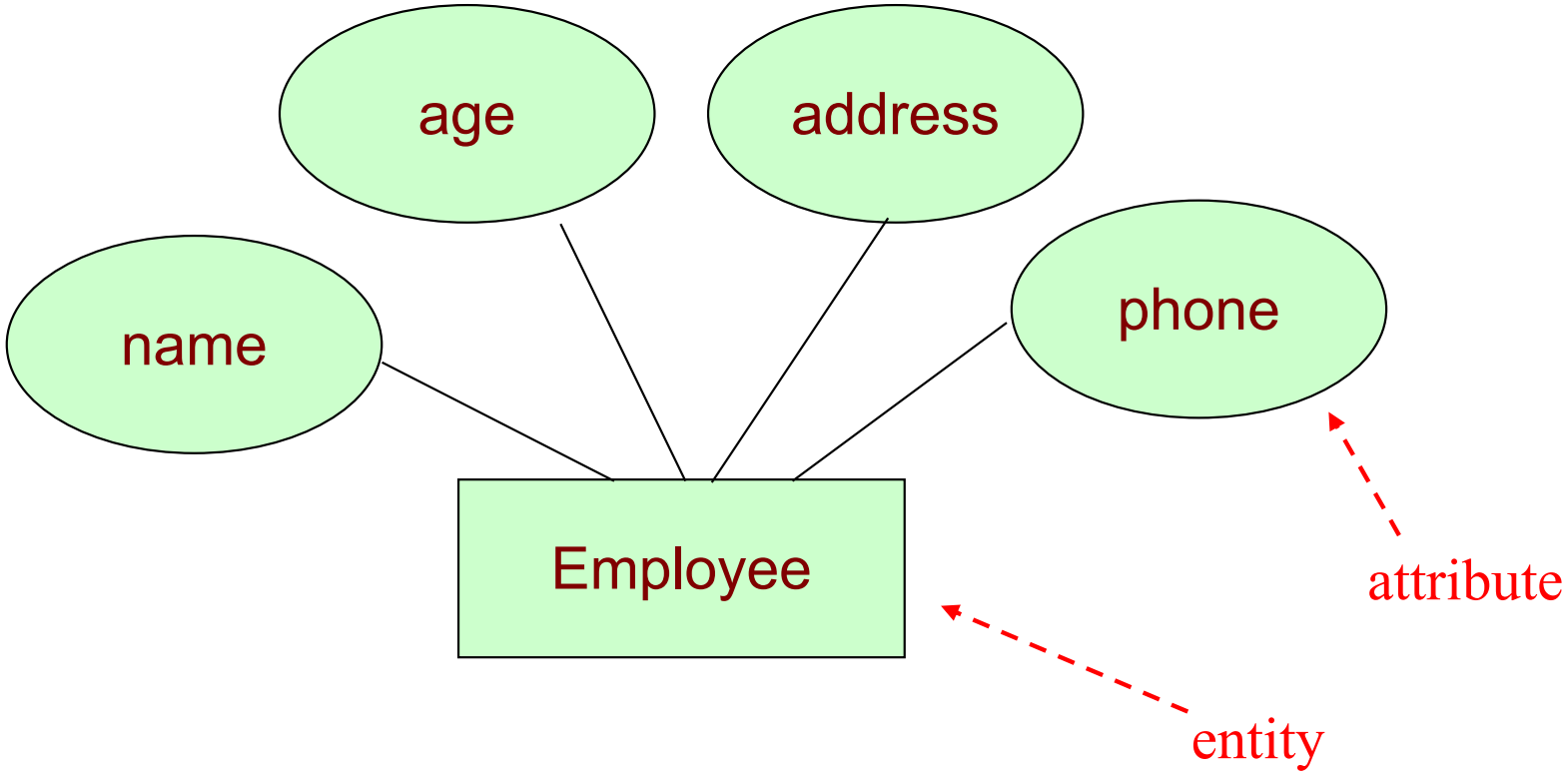
Entities and Attributes

- For each attribute associated with an entity set, we identify a domain of possible values.
- Example
  - The domain associated with the attribute name might be the set of 20-character *strings*.
  - The domain associated with the attribute age might be an *integer*.



ER Diagram

- The ER model can be presented graphically by an ER diagram
- Attribute: represented by a oval

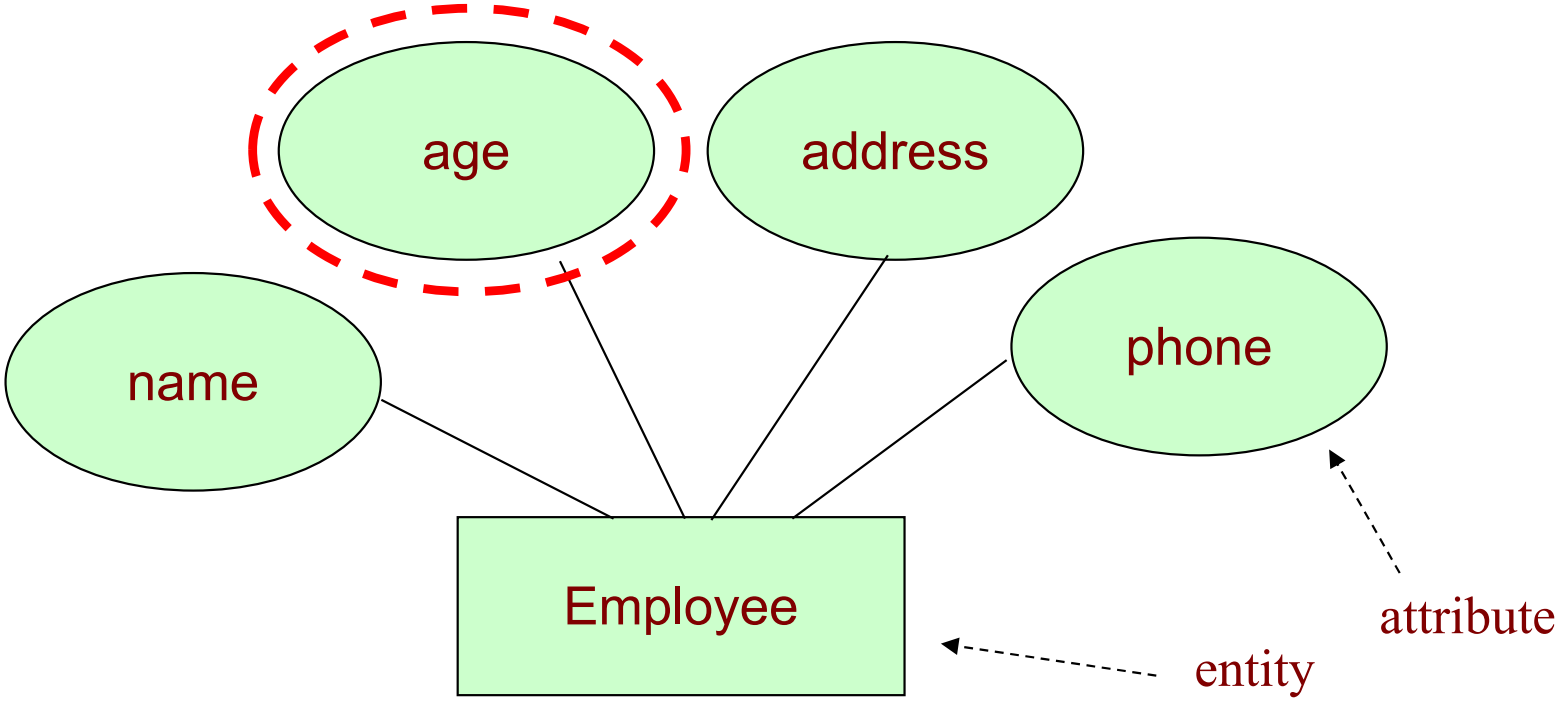


Different Attribute Types

- Simple attribute
- Composite attribute
- Multi-valued attribute
- Derived attribute

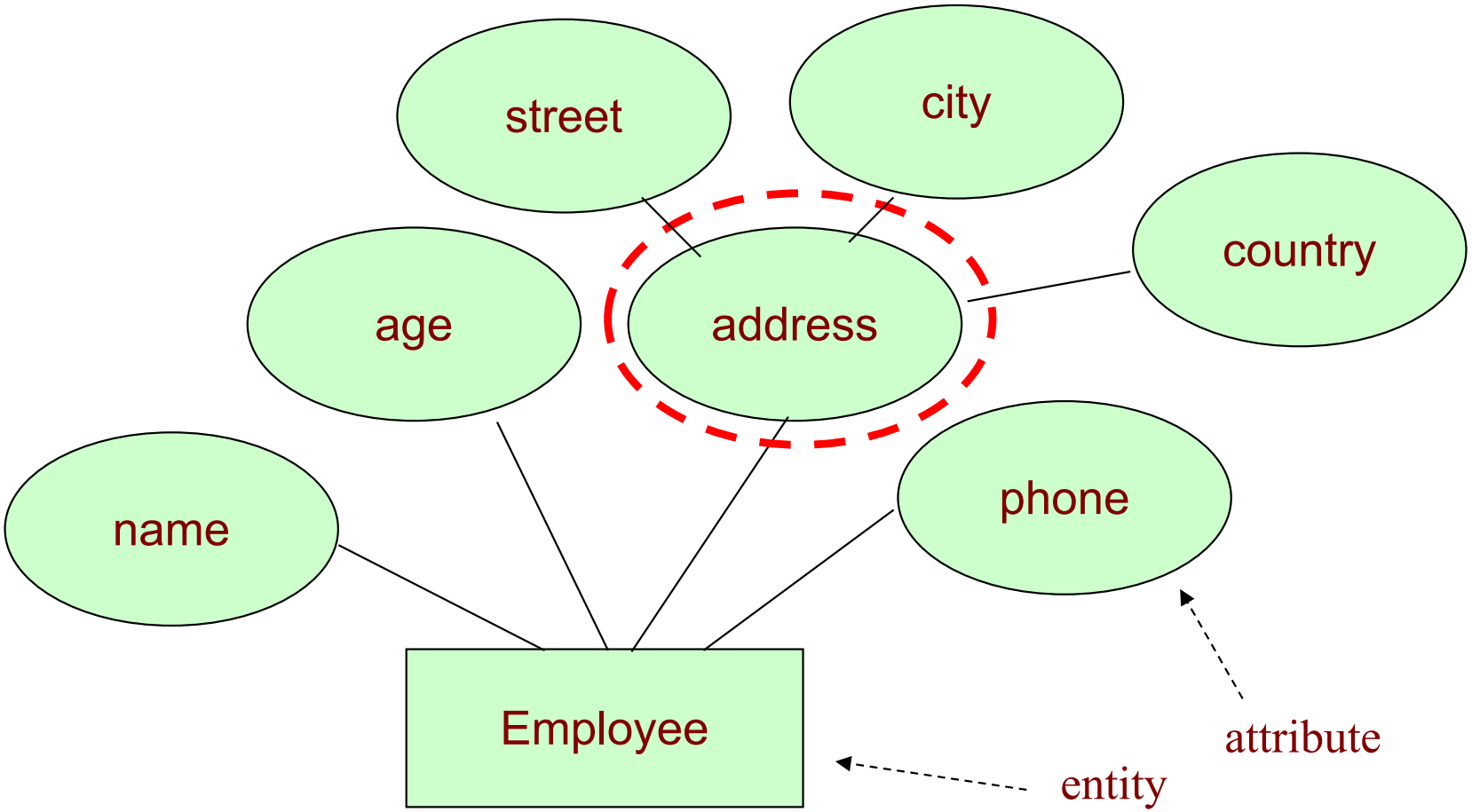
Simple attribute

- **Simple attribute**
  - contains a single value



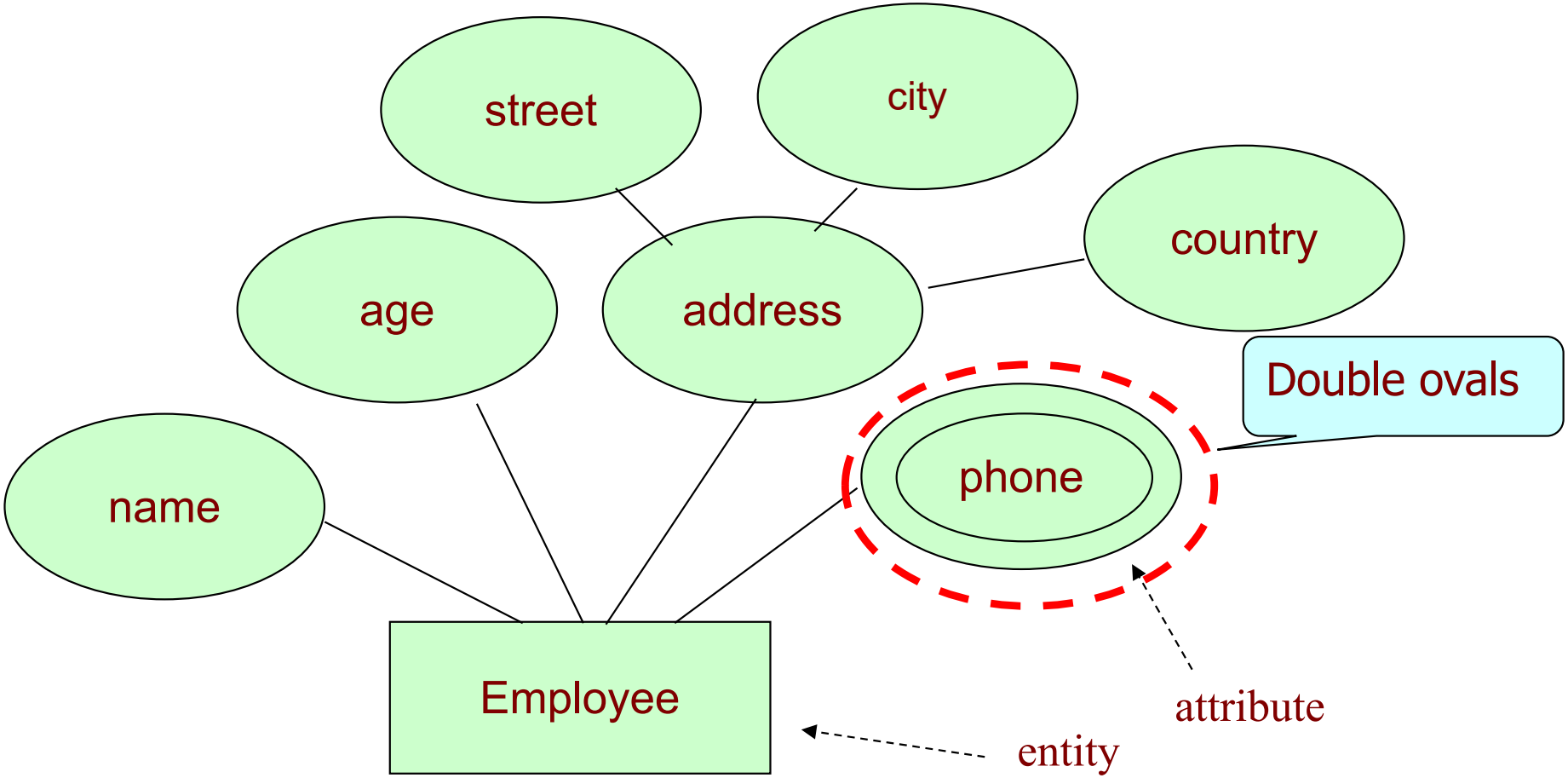
Composite attribute

- **Composite attribute**
  - Contains several components



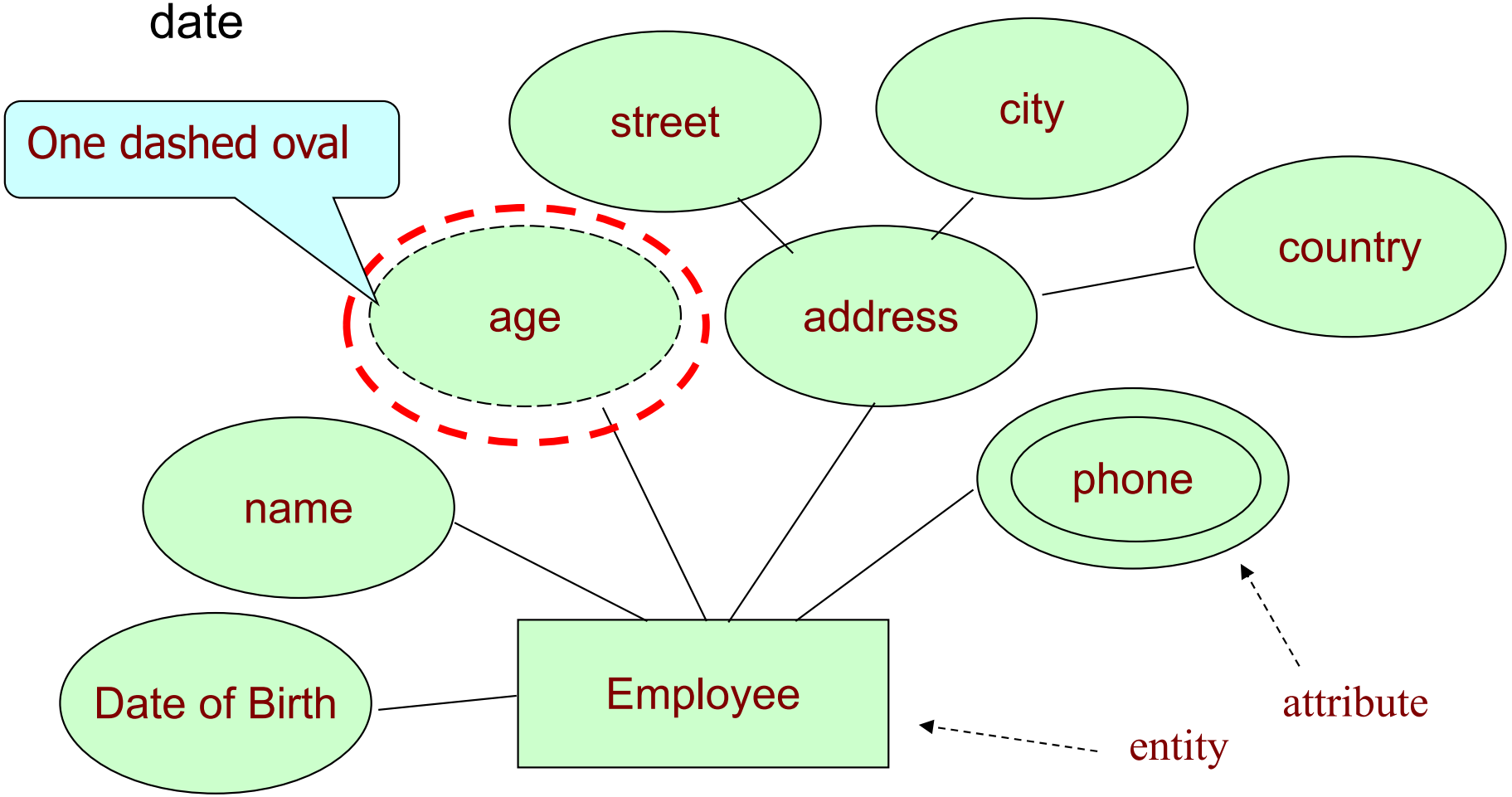
Multi-valued attribute

- **Multi-valued attribute**
  - Contains more than one value



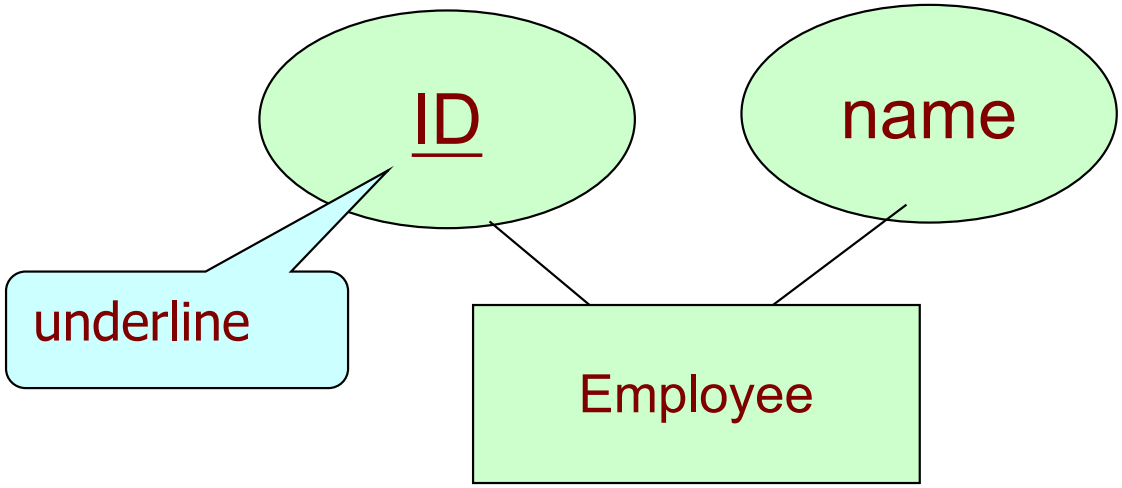
Derived attribute

- **Derived attribute**
  - Computed from other attributes
  - e.g., age can be computed from date of birth and the current date



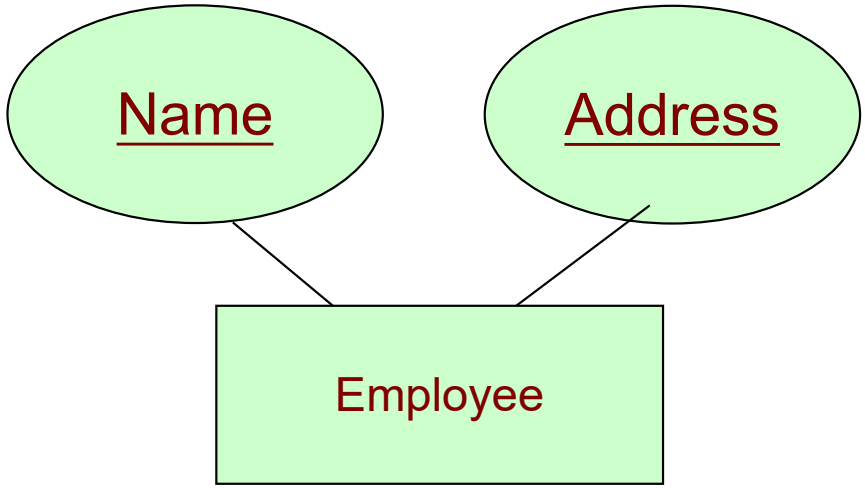
Key Attributes

- **Key**
  - A set of attributes that can **uniquely** identity an entity
  - E.g., ID



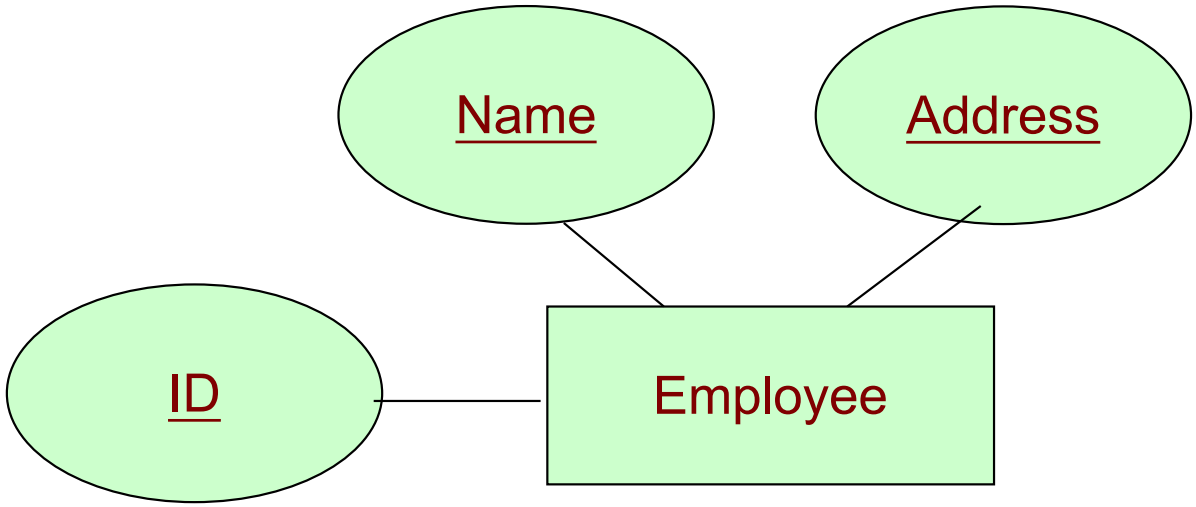
Key Attributes

- **Composite Key**
  - Two or more attributes are used to serve as a key
  - E.g., Name or Address alone cannot uniquely identify an employee, but together they can



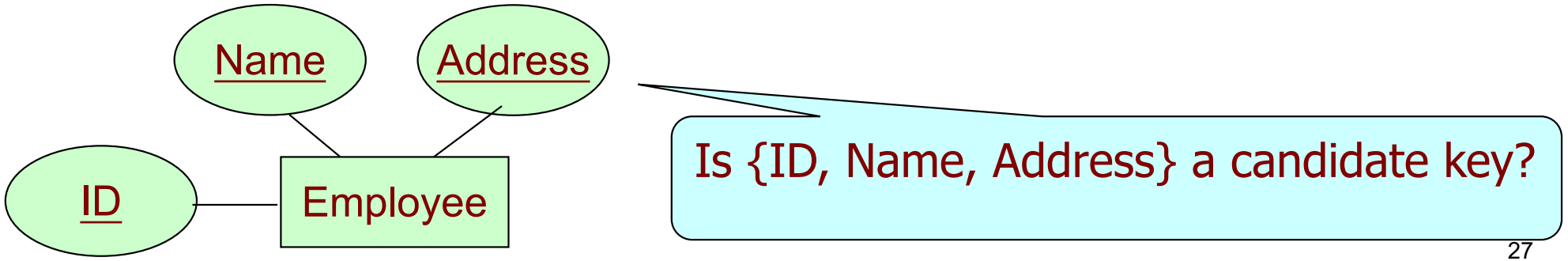
Key Attributes

- An entity may have more than one key
- E.g., {ID} and {Name, Address} both are two keys



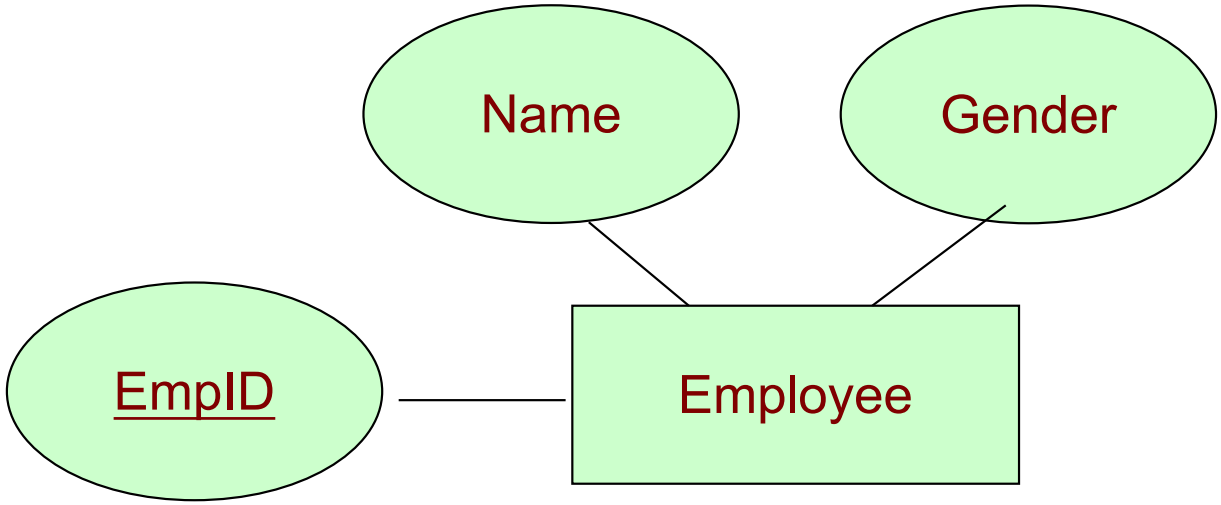
Key Attributes

- A **minimal** set of attributes that uniquely identifies an entity is called a **candidate key**.
- E.g., Are {ID} and {Name, Address} are two candidate keys?
- If there are many candidate keys, we should choose one candidate key as the **primary key**.



Key Attributes

- Sometimes, artificial keys can be created
- E.g., if there is no ID stored in Employee, we can create a new attribute called “EmpID”



Key Attributes

- The key should depend on the real life possibility rather than on the current set of the data.
- For example
  - In a database which contains only two employees aged 30 and 40, the age may distinguish each employee.
  - However, there can be in the future a new employee with the same age as an existing employee.
  - Therefore, {age} can not be a key.

Summary: Key

- ***Key***: An attribute or combination of attributes that uniquely identify an entity. A key means of specifying ***uniqueness***.
- ***Super Key***: A key that can be uniquely used to identify an entity, that may contain extra attributes that are not necessary to uniquely identify entities.
- ***Candidate Key***: A candidate key can be uniquely used to identify an entity without any extraneous data. It is a minimal super-key. Often abbreviate the ***Candidate Key*** to just ***Key***.

Summary: Key

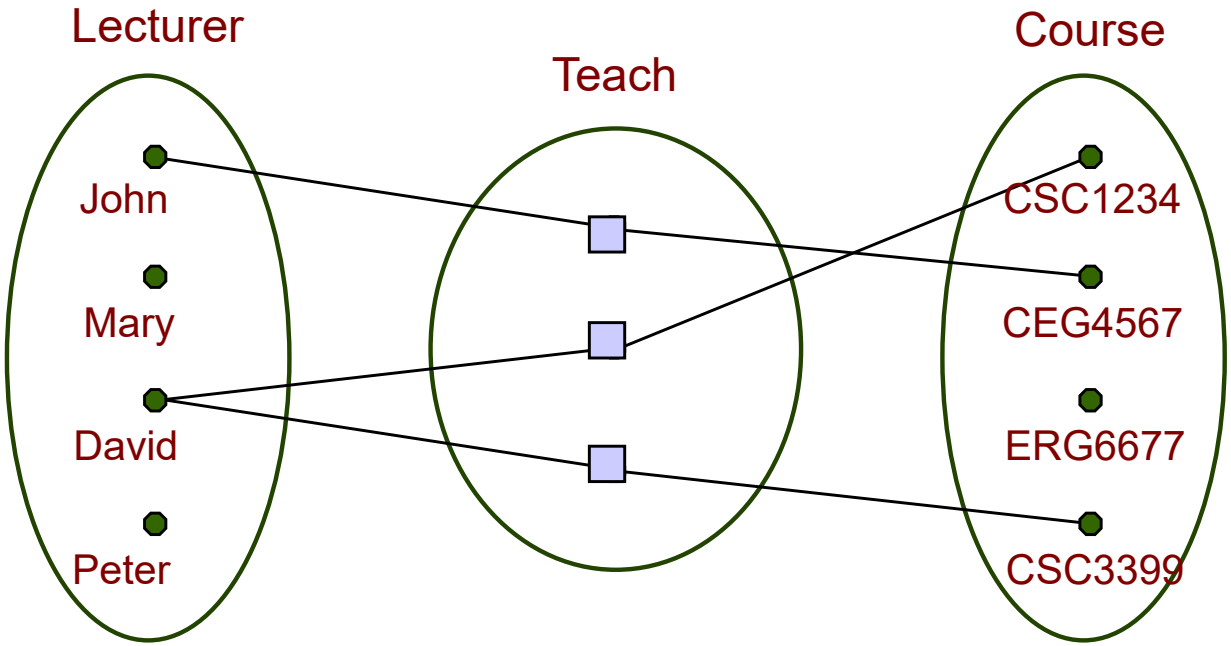
- ***Primary Key***: It is one of the candidate keys.
- ***Alternate Key***: A candidate key that is not the primary key is called an alternate key.
- ***Composite Key***: Key made up of multiple attributes.
- ***Artificial Key:*** It is the Surrogate primary key acting as the substitute of the primary key. It is generated artificial internally.

Outline

- Database Design
- Entity
- **Relationship**
  - **Binary relationship**
  - Non-binary relationship
- Weak Entity/Strong Entity
- Class Hierarchy
- Aggregation
- Conceptual Design with the E-R Model

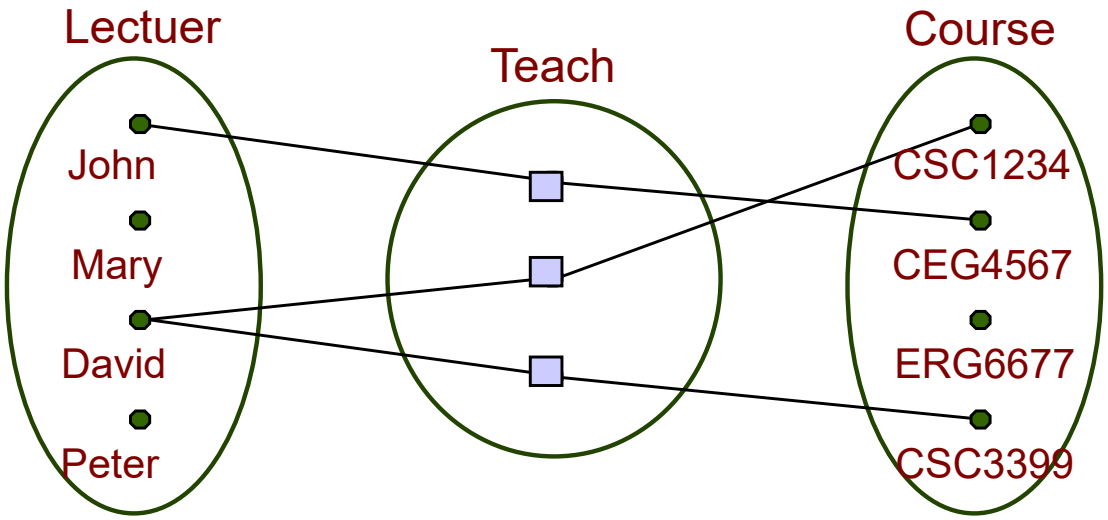
Relationships

- A ***relationship*** is an association among two or more entities.
- Example:
  - E1 = (John, Mary, David, Peter)
  - E2 = (CSC1234, CEG4567, ERG6677, CSC3399)



- As with entities, we may wish to collect a set of similar relationships into a ***relationship set***.
- A relationship set can be seen as a set of *n*-items (*n*-tuple):

$$\{(e_1,\cdots,e_n)\,|\,e_1 \in E_1,\cdots e_n \in E_n\}$$



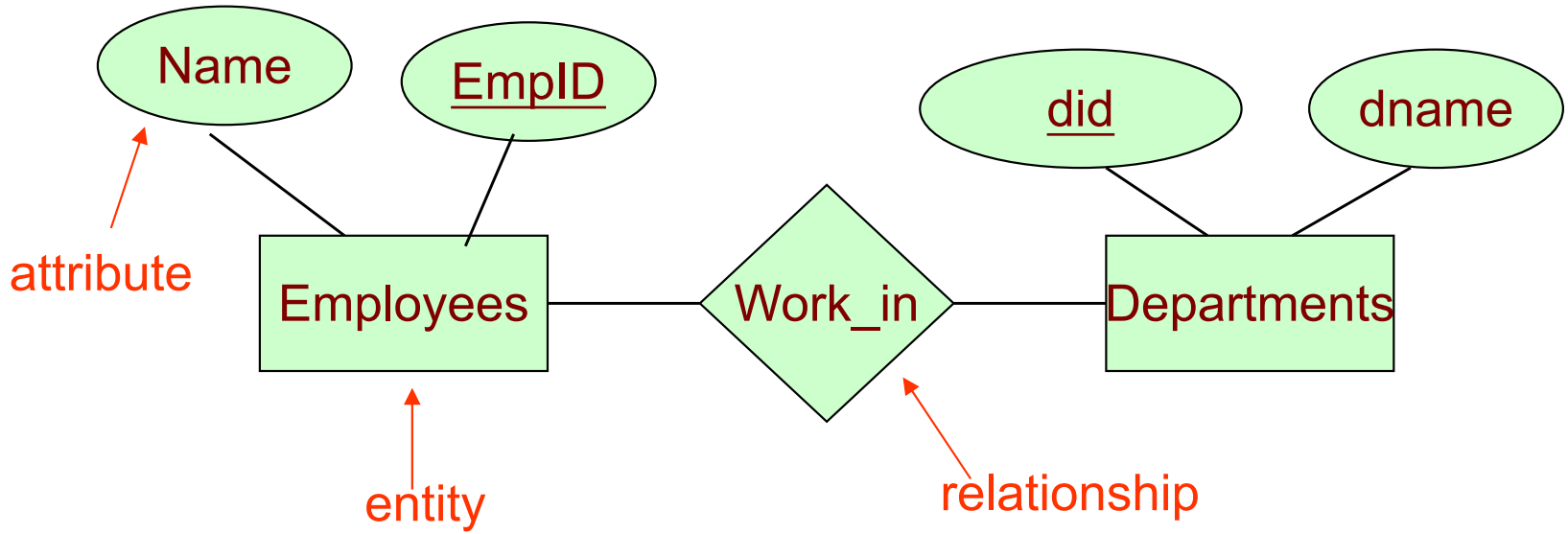
- The teach relationship can be represented by the set of ordered pairs: {(John,CEG4567), (David, CSC1234),(David,CSC3399)}

Relationships

- The ***degree*** refers to the number of entity sets that participate in a relationship set.
- Relationship sets that involve two entity sets are ***binary*** (or degree two).
- Relationships among more than two entity sets are rare. (We will discuss later in details)

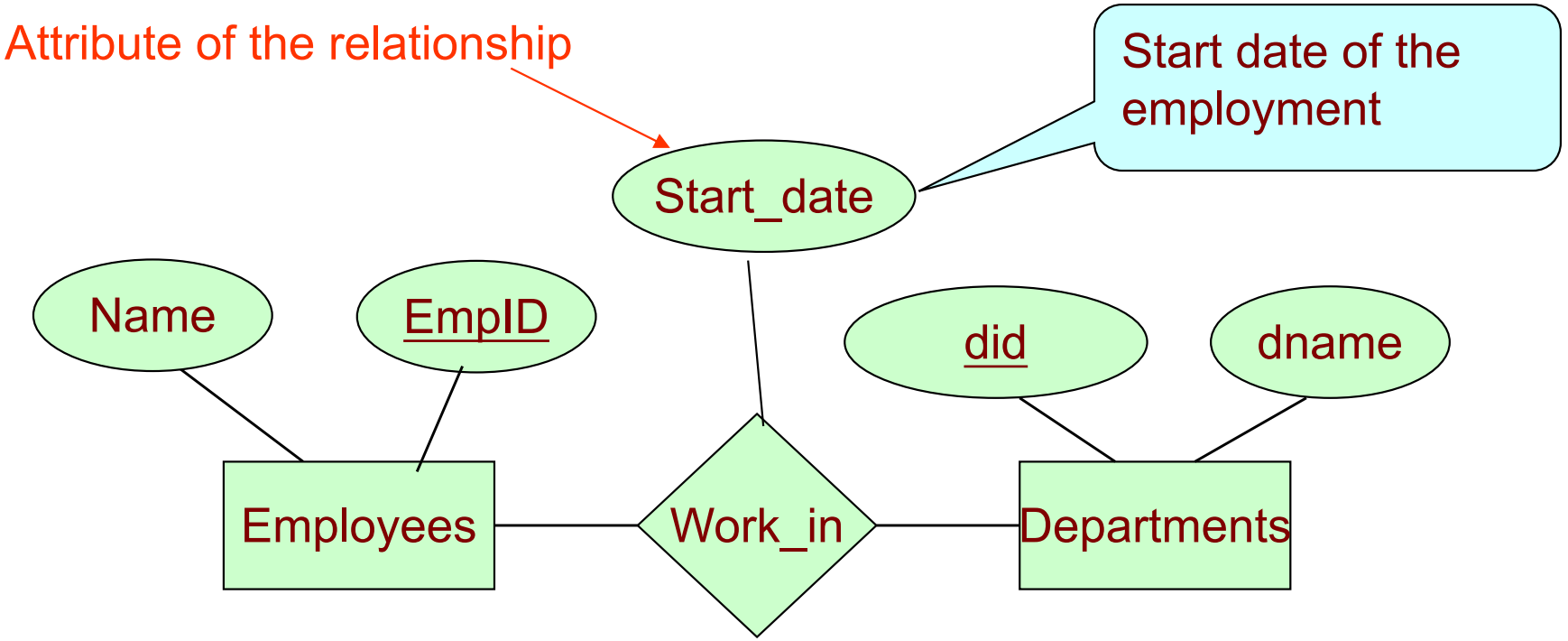
Binary Relationship

- Employees work in departments
- “Work_in” is a relationship between Employees and Departments
- It means that an employee works in multiple departments and vice versa

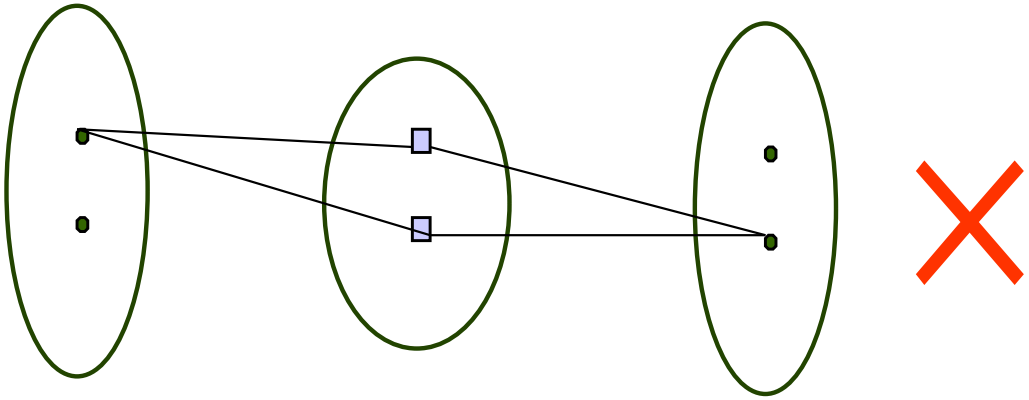


Binary Relationship

- A relationship can also have attributes which are used to describe the record information about the relationship (instead of the information of each individual entity).

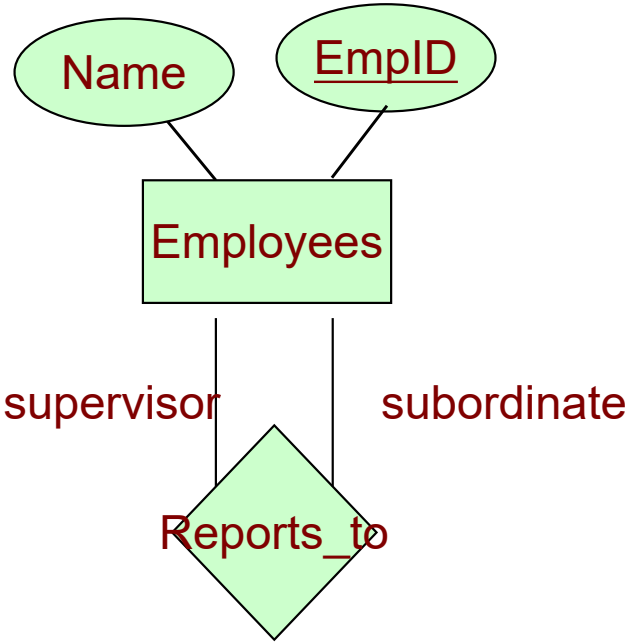


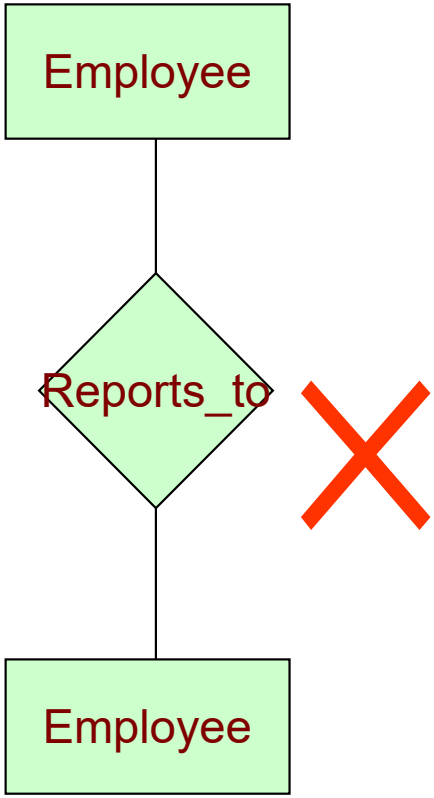
- A relationship must be uniquely identified by the participating entities, without reference to the descriptive attributes.
  - In the previous example, each relationship must be uniquely identified by the combination of the *employee id* and the *department did*.
- Thus, for each employee-department pair, we cannot have more than one associated “start_date” value.
  - (Mary-ID, Dept ABC ; **1-2003**)
  - (Mary-ID, Dept ABC ; **2-2004**)



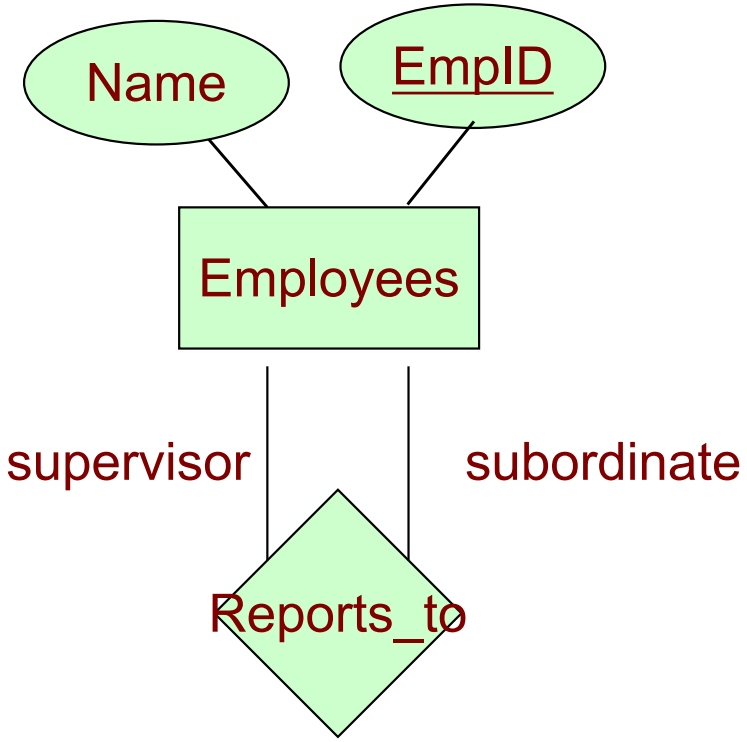
Recursive Relationship

- Recursive Relationship
  - Entity sets of a relationship need not be distinct
  - Sometimes, a relationship might involve two entities in the same entity set
  - E.g., Employees related to employees





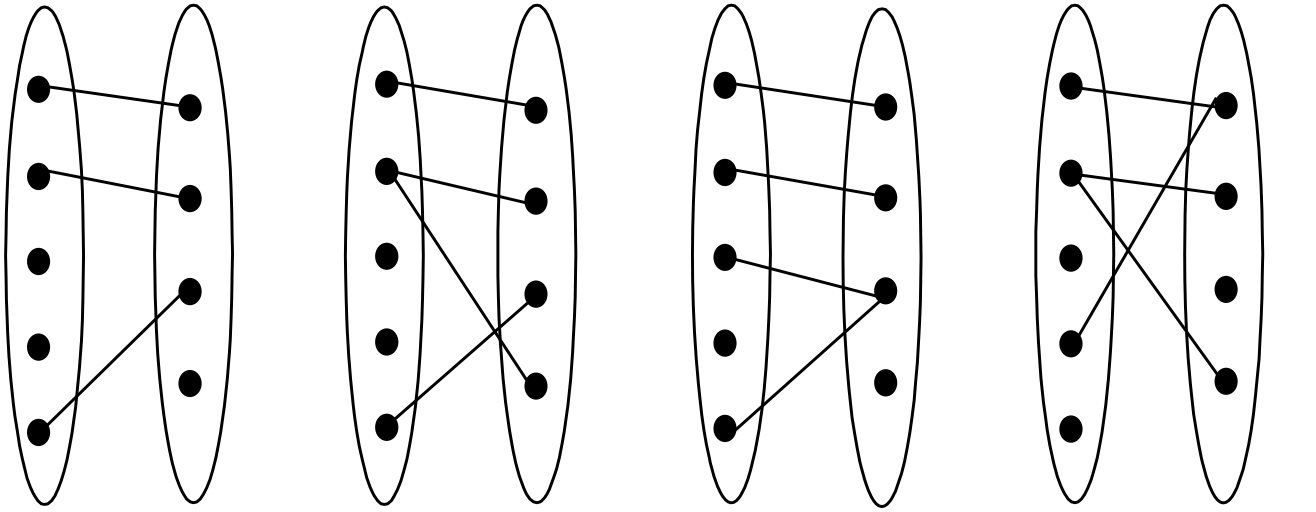
Recursive Relationship



- Since employees report to other employees
  - Every relationship in “Reports_To” is of form (emp1, emp2) where both emp1 and emp2 are entities in employees.
  - However, they play different roles.
    - emp1 reports to emp2, which is reflect in the role indicators supervisor and subordinate in the diagram

Constraints

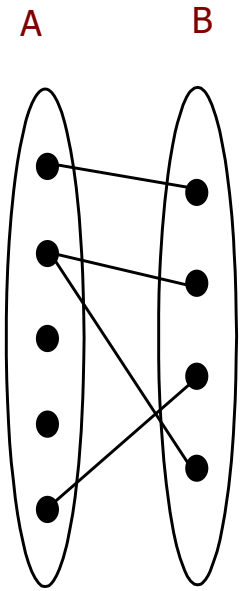
- The model describes data to be stored and the **constraints** over the data
- The type of association of a binary relationship can be classified into the following cases:



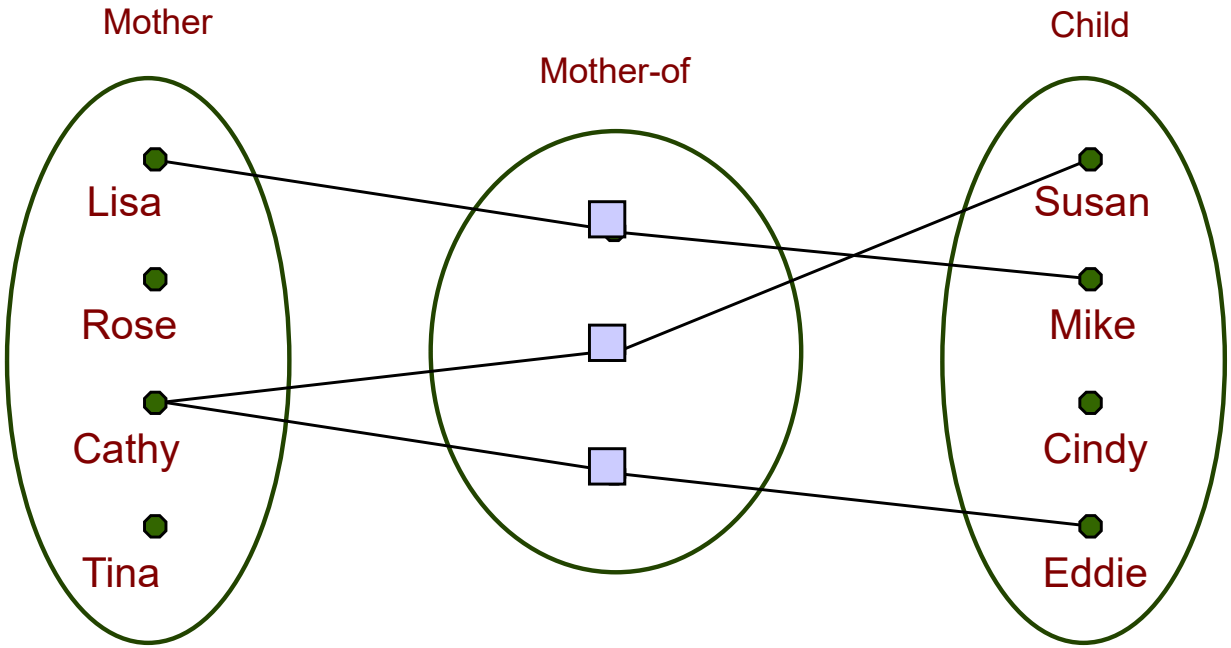
The preceding discussion identified each relationship in both directions; that is, relationship are ***bidirectional***.

One-to-many relationship

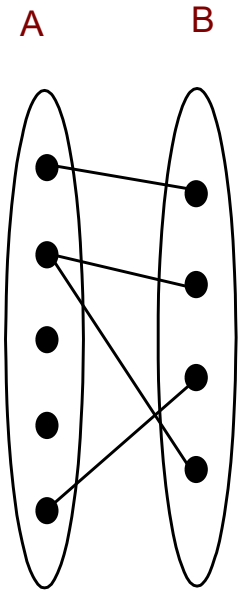
- An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A.
- Example: each child can appear in at most one mother-child relationship.



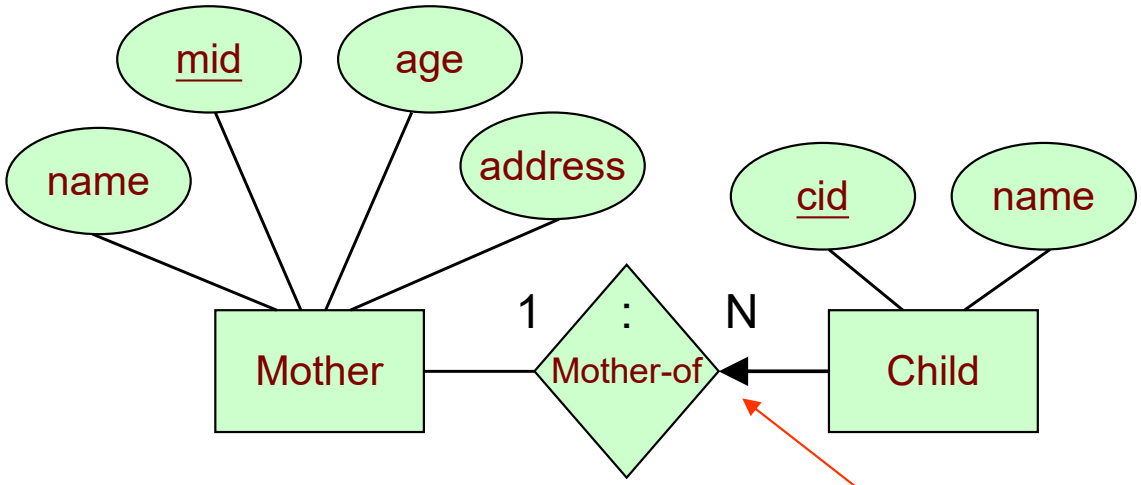
1-to-Many
1:N



- Child has a key constraint in the mother-of relationship set.
- This restriction can be indicated by an arrow in the E-R diagram.
  - **A** Child is associated with at most **one** Mother via Mother-of
  - **A** Mother is associated with **several** (including 0) Children via Mother-of



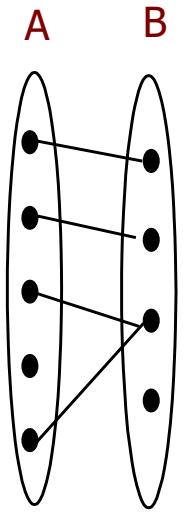
1-to-Many
1:N



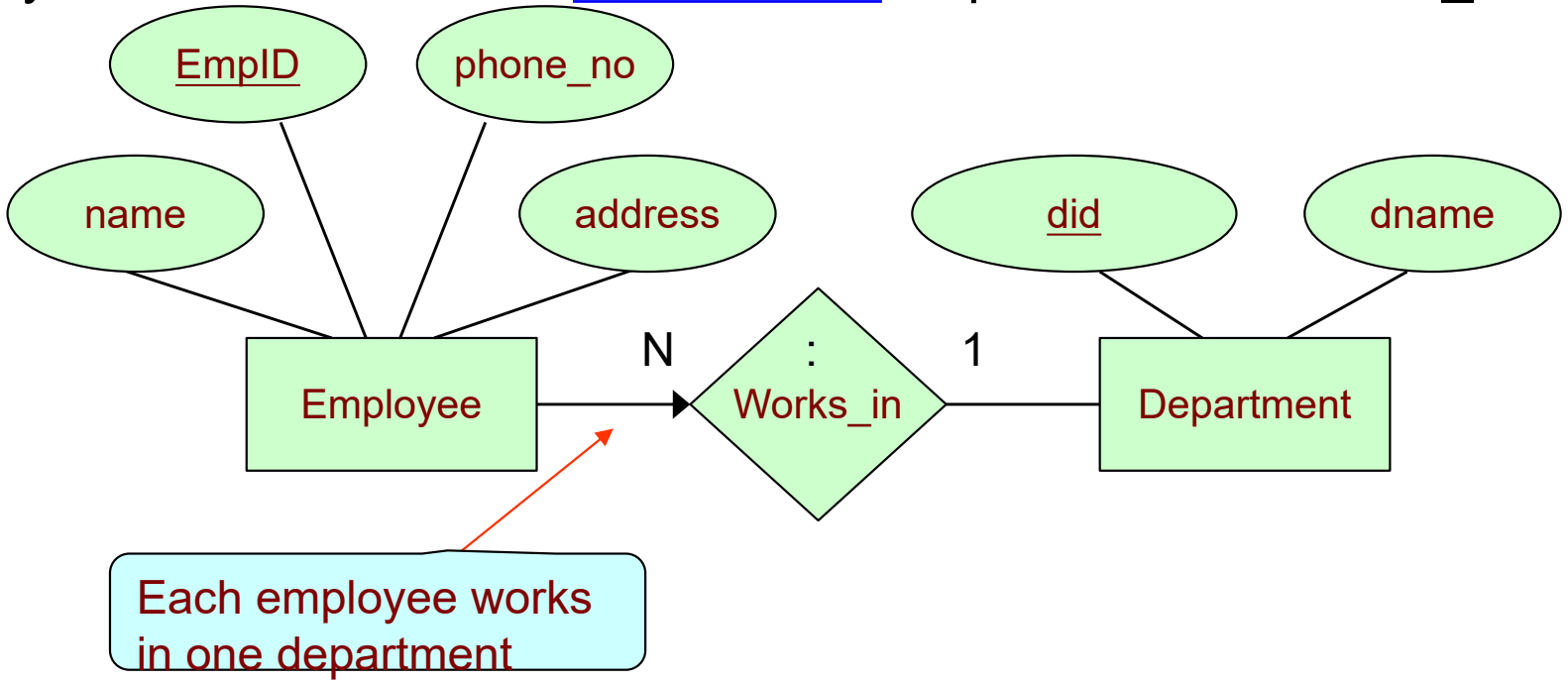
Intuitively, the arrow states that given a child entity, we can uniquely determine the mother-of relationship.

Many-to-one relationship

- An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A.
- Example:
  - **A** Department is associated with **several** (including 0) Employees via Works_in,
  - **A** Employee is associated with **at most one** Department via Works_in

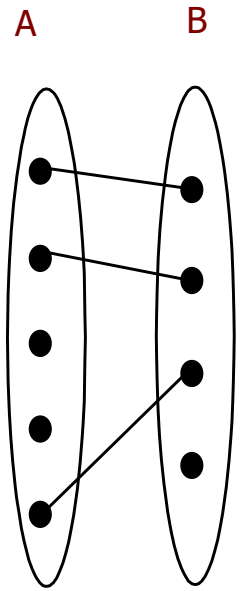


Many-to-1
N:1

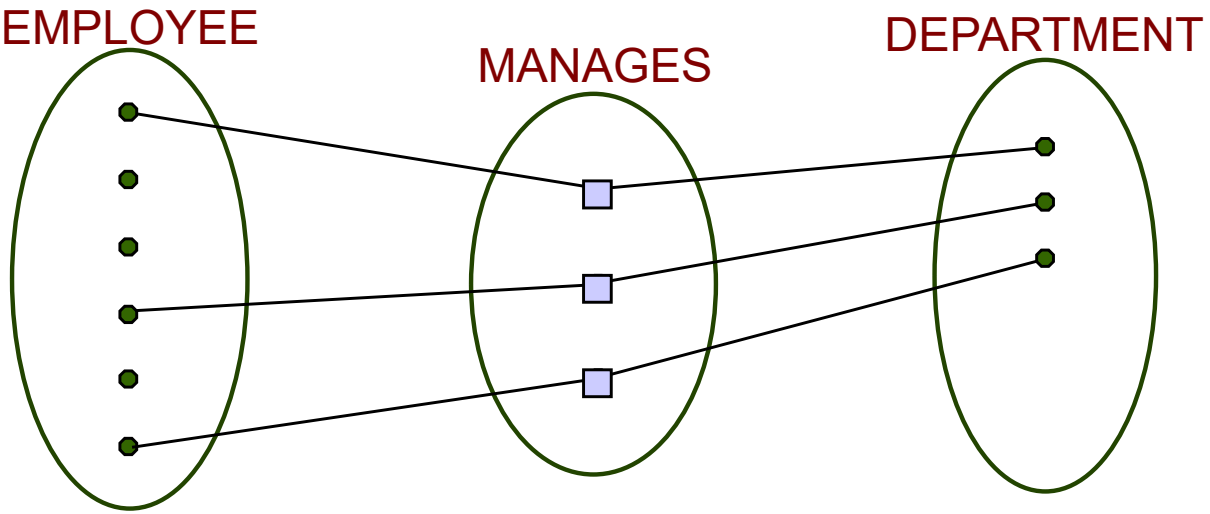


One-to-one relationship

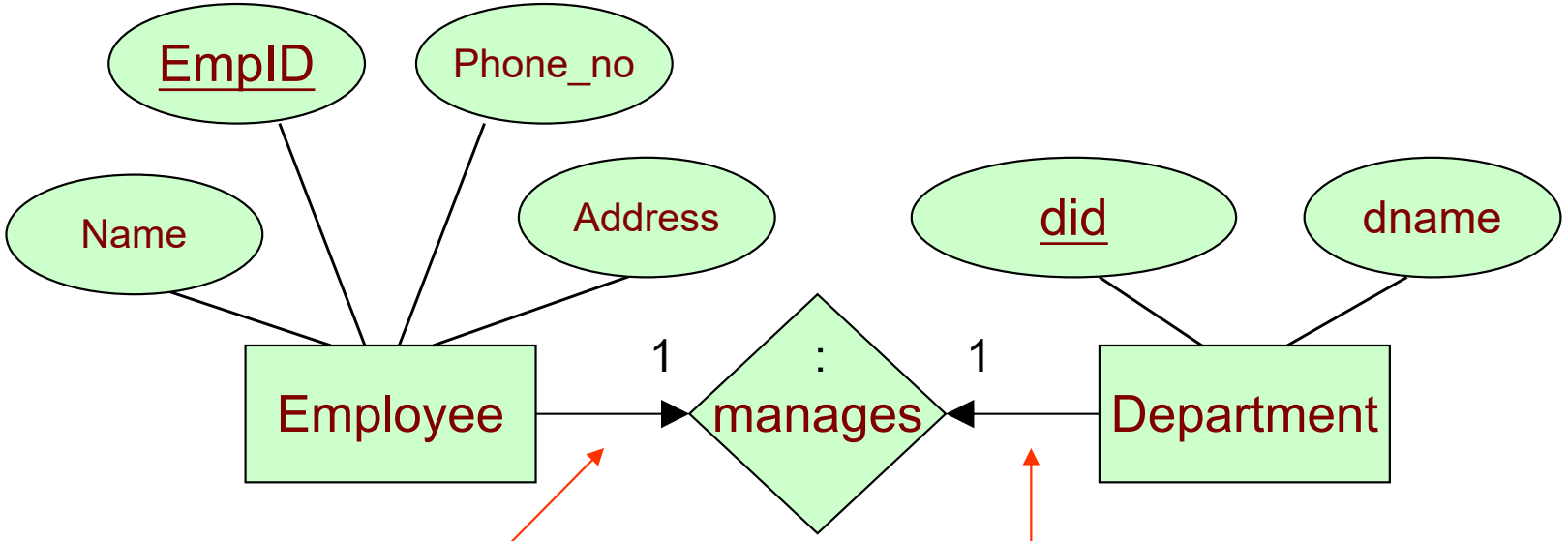
- If the relationship between A and B satisfies the one-to-one mapping constraint from A to B, then
- **An** entity in A is related to **at most one** entity in B, and
- **An** entity in B is related to **at most one** entity in A.



1-to-1
1:1

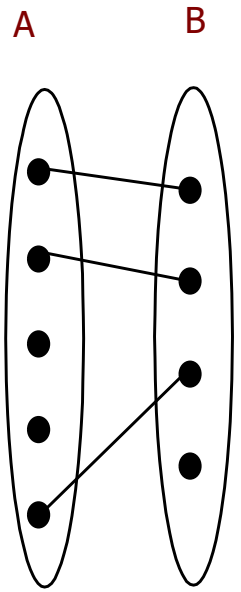


- A Employee is associated with at most one Department via the relationship manages
- A Department is associated with at most one Employee via manages



An employee can associate with at most one department via the Relationship “manages”.

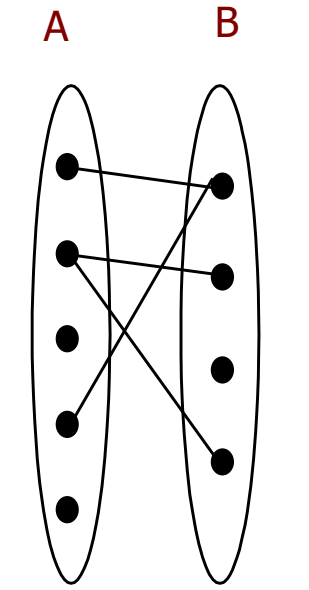
A department can associate with at most one employee via the relationship “manages”



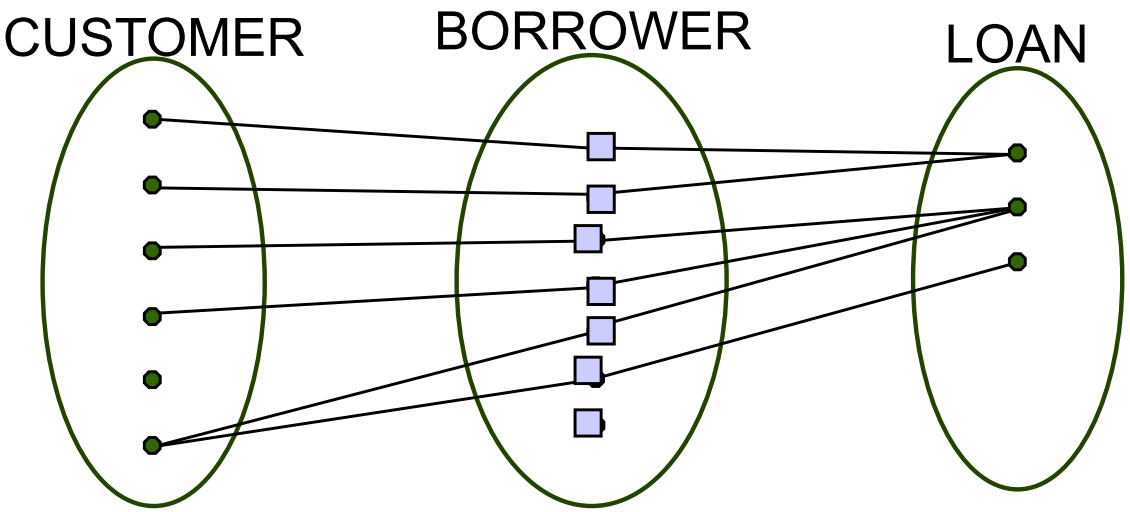
1-to-1
1:1

Many-to-many Relationship

- **An** entity in A is associated with **any number** of entities in B
- **An** entity in B is associated with **any number** of entities in A
- In fact, there is no restriction in the mapping.

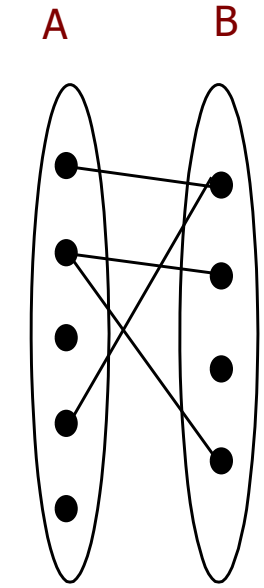


Many-to-Many
N:M

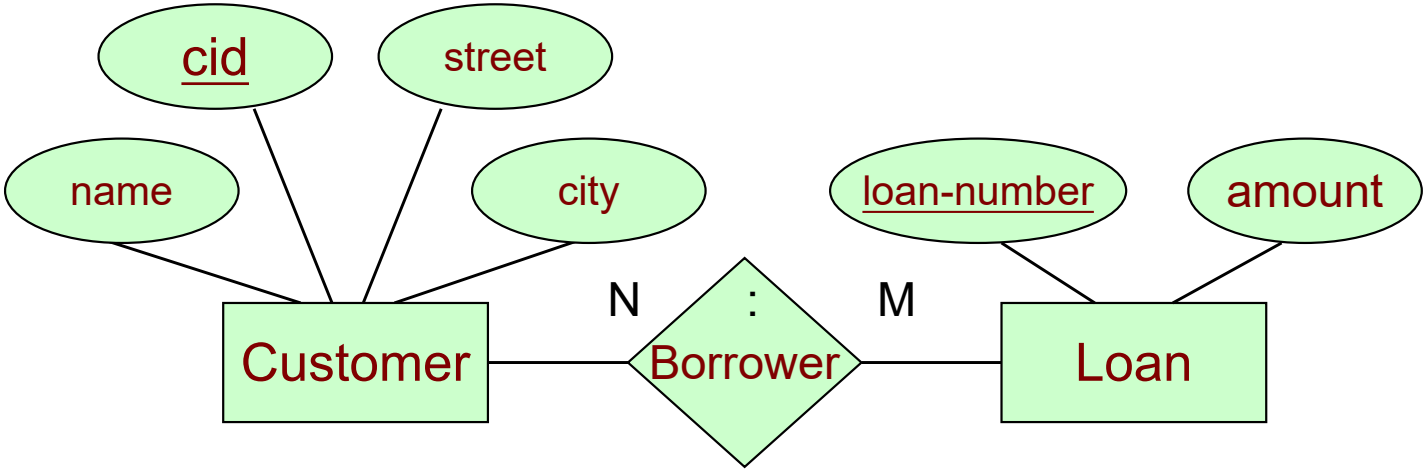


Many-to-many Relationship

- A customer can associate with several loans (possibly 0) via Borrower
- A loan can associate with several customers (possibly 0) via Borrower



Many-to-Many
N:M



Keys for a Relationship Set

- The concept of keys is also used to identify a relationship, as in entities.
- Key of relationship set R
  - A set of attributes that can ***uniquely*** identify a relationship in R.
- Let R be a relationship set involving entity sets E1, E2, …, En. Let *primary-key(Ei)* denote the set of attributes that forms the primary key for entity set Ei. The set of attributes

  primary-key(E1) ∪ *primary-key(E2)* ∪ □ ∪ *primary-key(En)*

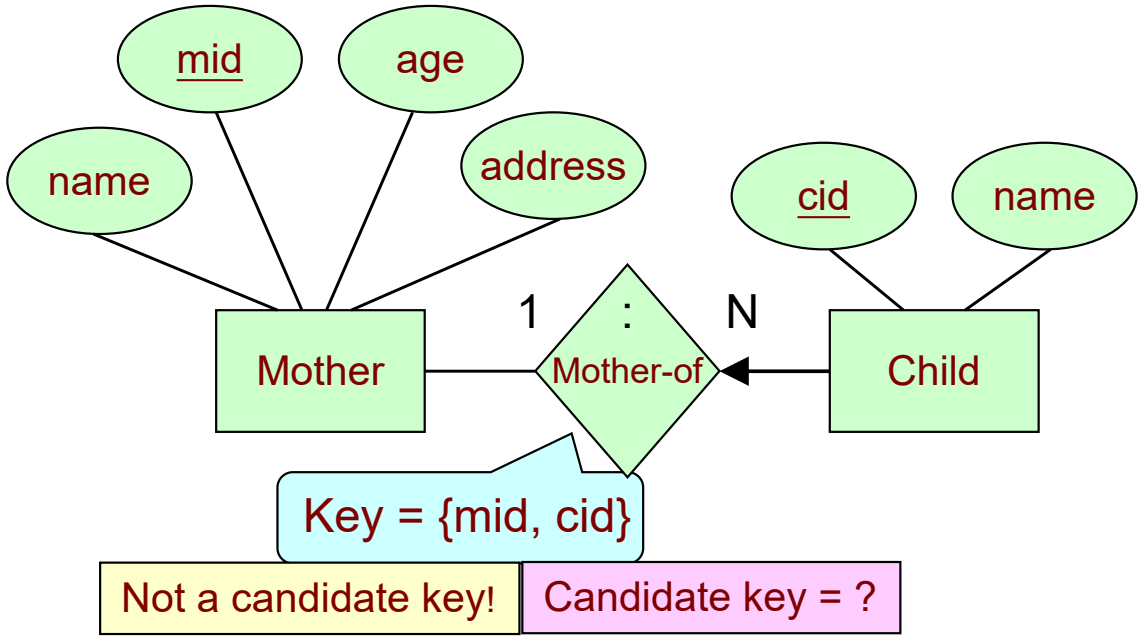
  forms a ***superkey*** for the relationship set.

Keys for a Relationship Set

- The choice of the ***primary key*** for a binary relationship set depends on the mapping cardinality of the relationship set.
  - One-to-many and many-to-one relationships, the primary key of the "***many***" side is a minimal superkey and is used as the ***primary key***.
  - One-to-one relationships, ***the primary key of either one*** of the participating entity sets forms a minimal superkey, and either one can be chosen as the ***primary key*** of the relationship set.
  - Many-to-many relationships, ***the union of the primary keys*** is a minimal superkey and is chosen as the ***primary key.***

Key Constraints (1-to-many)

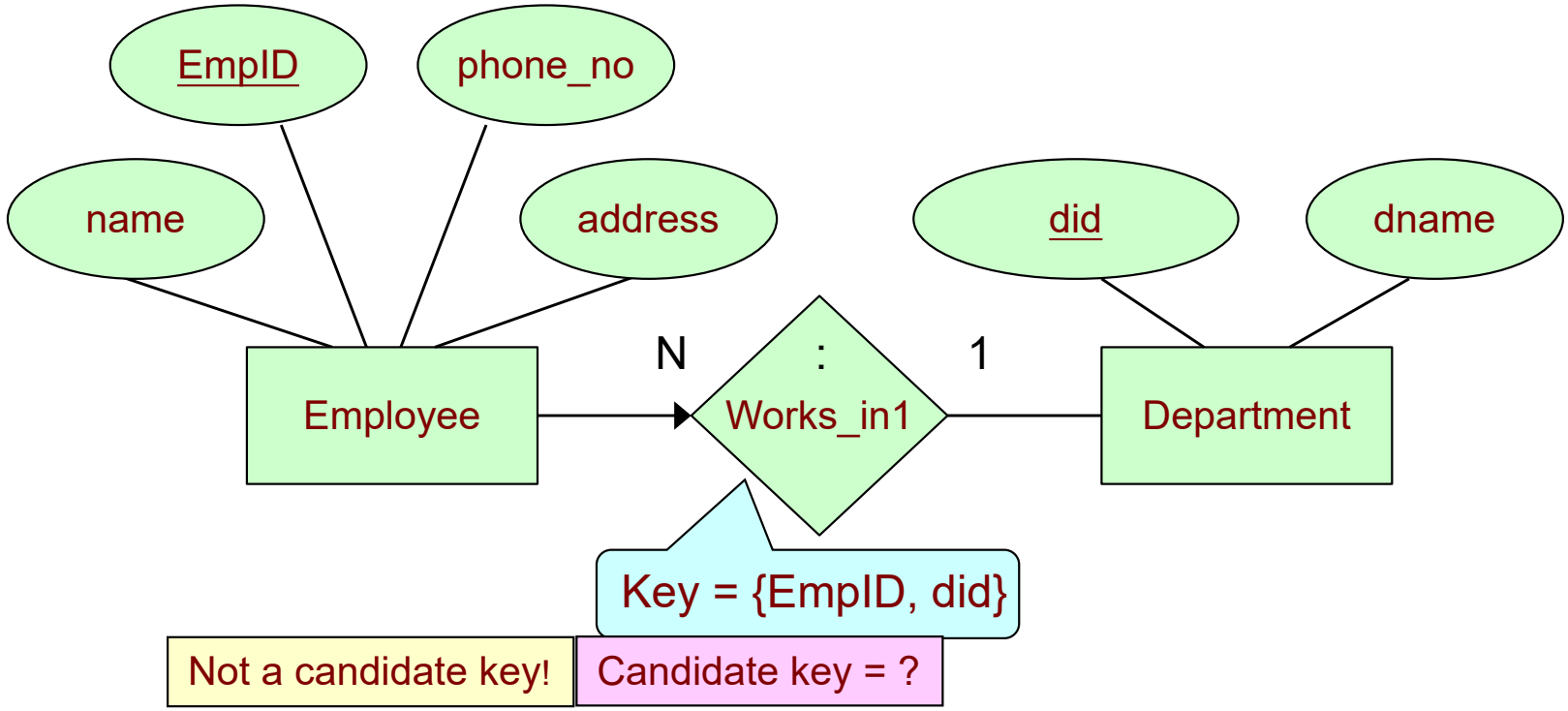
- Child has a key constraint in the mother-of relationship set.



Primary Key for Mother-of = (cid)

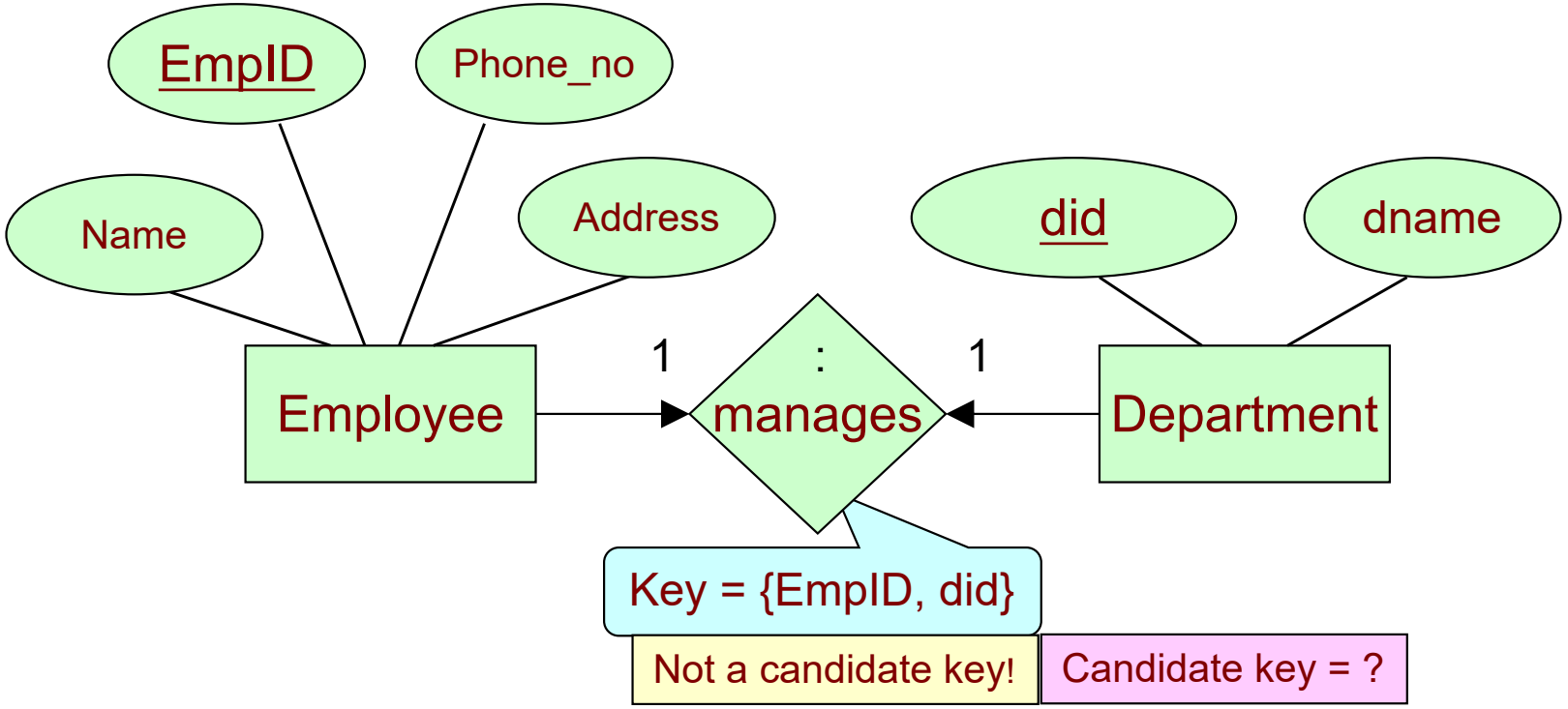
Key Constraints (many-to-1)

- **A** Department is associated with **several** (including 0) Employees via Works_in,
- **A** Employee is associated with **at most one** Department via Works_in



Key Constraints (1-to-1)

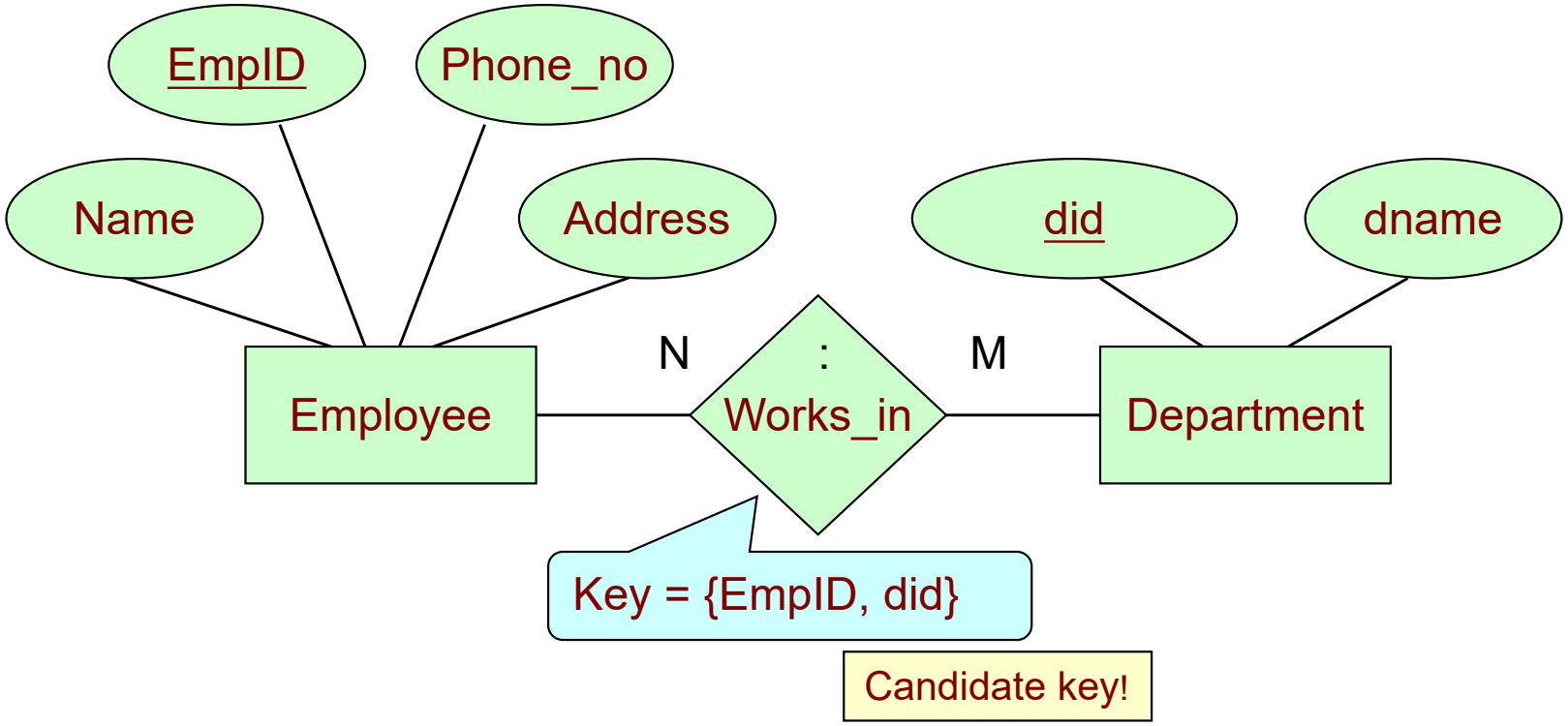
- A Employee is associated with at most one Department via the relationship manages
- A Department is associated with at most one Employee via manages



Primary Key for manages = (EmpID) Or (did)

Key Constraints (many-to-many)

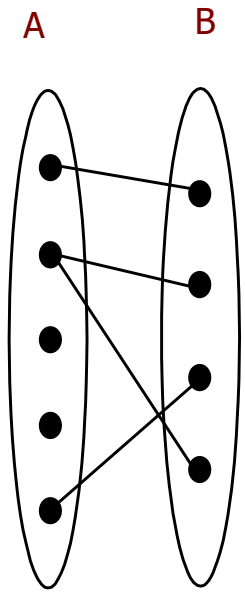
- A employee can associate with several departments (possibly 0) via Works_in
- A department can associate with several employees (possibly 0) via Works_in



Primary Key for Works_in = (EmpID, did)

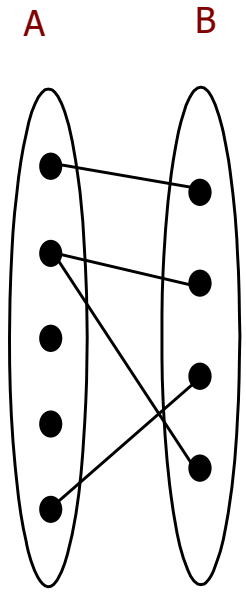
Participation Constraint

- The above constraints (e.g., Works_in) tells us that a department have some employees.
- A natural question to ask is whether every department at least have one employee.
- Suppose that each department at least have one employee. Such a constraint is called a **participation constraint.**



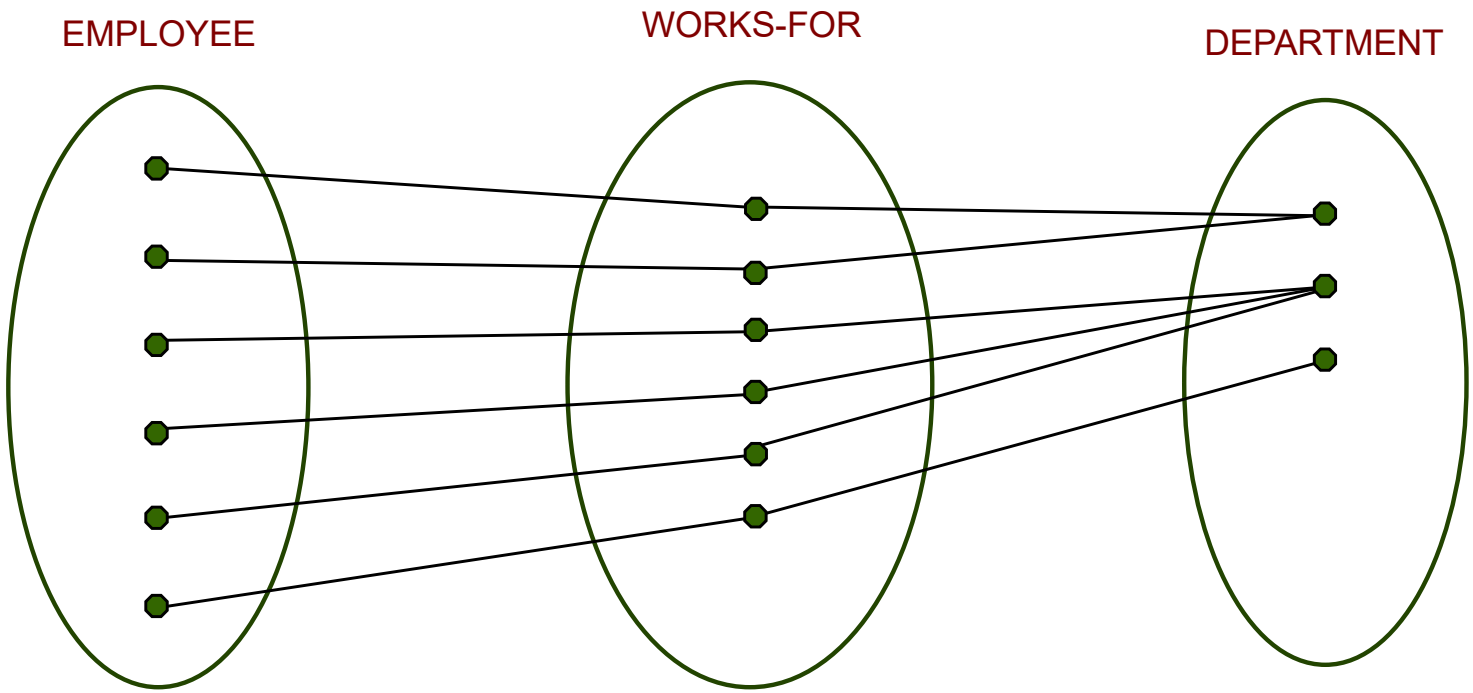
Participation Constraint

- We can classify participation in relationships as follows.
- **Total**
  - Each entity in the entity set must be associated in at least one relationship
- **Partial**
  - Each entity in the entity set may (or may not) be associated in a relationship

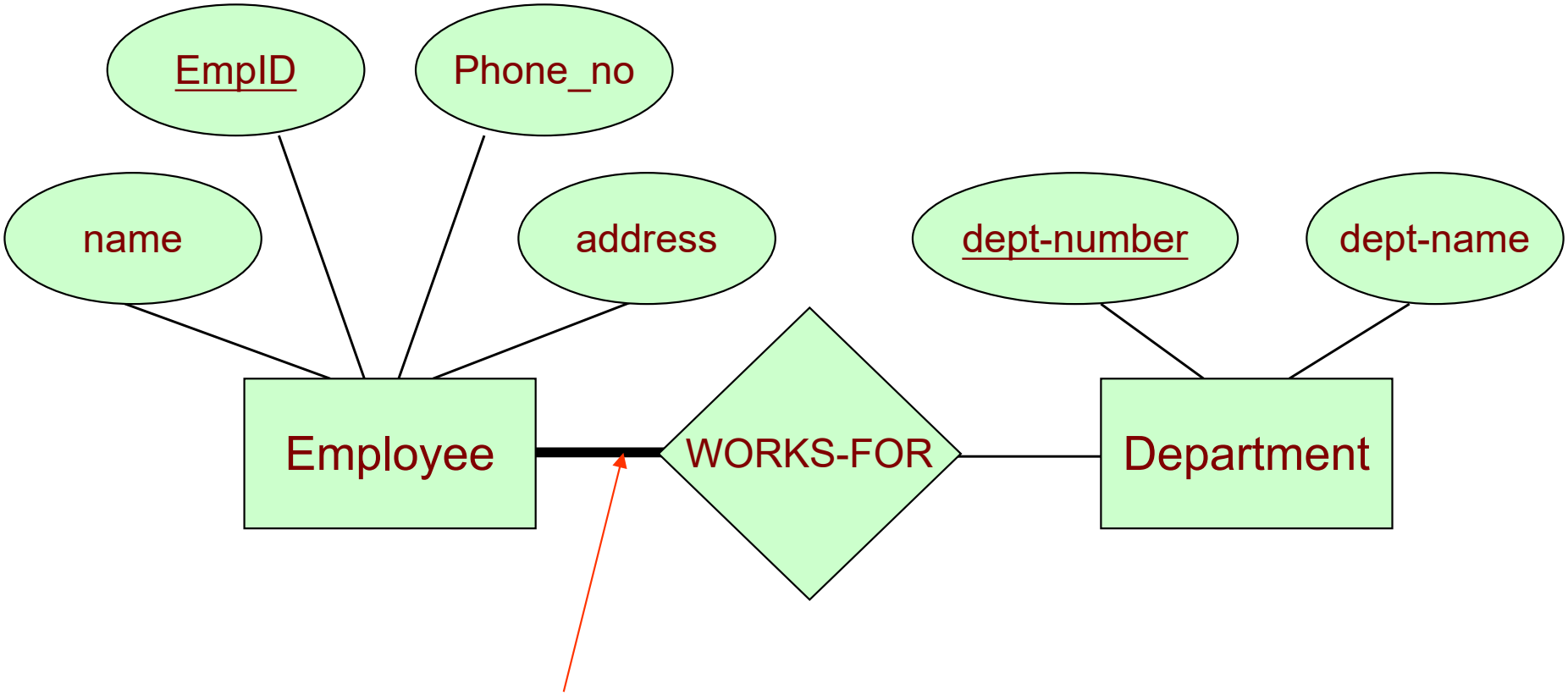


Participation Constraint

- For example
  - ***Every employee*** must work for some department.
  - The participation of EMPLOYEE in WORKS-FOR is ***total participation***



Participation Constraint



If the participation of an entity set in a relationship set is total, the two are connected by a thick link.
Independently, the presence of an arrow indicates a key constraint.

Outline

- Database Design
- Entity
- **Relationship**
  - Binary relationship
  - **Non-binary relationship**
- Weak Entity/Strong Entity
- Class Hierarchy
- Aggregation
- Conceptual Design with the E-R Model

Non-Binary Relationship Set

- A relationship set is a subset of the Cartesian product

$$E_1 \times E_2 \times \ldots \times E_n$$

- *n* = the degree of the relationship set
- In general, a non-binary relationship set cannot be directly replaced by a number of binary relationship sets.

Example: Ternary Relationship

- Suppose that each department has offices in several locations and we want to record the locations at which each employee works.